

OpenAI AgentKit — Course Reference

From the Responses API primitive to the AgentKit product stack

Course: Building Agents with OpenAI's AgentKit & Responses API (Lucas Soares, O'Reilly Live Training) | **Updated:** June 2026 | **Models:** `gpt-5.4-mini` (default), `gpt-5.5` (frontier)

The full OpenAI agent stack builds in layers: the **Responses API** is the primitive (one model call with server-side state and built-in tools); the **Agents SDK** adds orchestration; **AgentKit** (Agent Builder, ChatKit, Connector Registry, Evals) is the product platform on top. Learn the primitive first — everything else composes from it.

1. The Stack at a Glance

Layer	What it is	You reach for it when...
Responses API	Single stateful model call: items I/O, server-side state, built-in tools, structured outputs	You want direct control over one model turn
Conversations API	Durable, server-side conversation object the Responses API writes to	You need persistent multi-turn state without resending history
Agents SDK	Framework: multi-agent orchestration, handoffs, guardrails, tracing (100+ LLMs)	You need multiple agents / tools / routing in code
Agent Builder	Visual drag-and-drop workflow composer (nodes, classifiers, guardrails)	You want to compose a workflow without code (<i>deprecated Nov 30 2026 — patterns transfer to the SDK</i>)
ChatKit	Embeddable React/Web-Component chat UI (streaming, file upload, tool display)	You want a ready chat front-end for a workflow
Guardrails	Open-source safety layer: PII detection, jailbreak & prompt-injection defense	You need input/output safety around an agent

2. Responses API — the primitive

2.1 Basic call & output

```
from openai import OpenAI
client = OpenAI() # reads OPENAI_API_KEY

resp = client.responses.create(
    model="gpt-5.4-mini",
    input="Explain agents in one sentence.",
)
print(resp.output_text) # convenience accessor for the text
# resp.output is the list of output items (reasoning, tool calls, message)
```

2.2 Input items & multi-turn

Inputs are a list of typed items. Chain turns server-side with `previous_response_id` (no need to resend history).

```
resp = client.responses.create(
    model="gpt-5.4-mini",
    input=[{"role": "user", "content": "What are 3 core snowboarding skills?"}],
)
followup = client.responses.create(
    model="gpt-5.4-mini",
    input=[{"role": "user", "content": "Give 3 drills for each."}],
    previous_response_id=resp.id,    # server keeps the prior context
)
```

2.3 Structured outputs (Pydantic)

Use `responses.parse` with a Pydantic model as `text_format`; read the typed object from `output_parsed`.

```
from pydantic import BaseModel

class PaperData(BaseModel):
    title: str
    abstract_summary: list[str]

resp = client.responses.parse(
    model="gpt-5.4-mini",
    input=[{"role": "user", "content": paper_text}],
    text_format=PaperData,
)
data = resp.output_parsed          # -> PaperData(title=..., abstract_summary=[...])
```

2.4 Token counting (before you send)

Ask the API for the exact input-token count for a given model — more reliable than a local tokenizer for new models.

```
result = client.responses.input_tokens.count(
    model="gpt-5.5",
    input="Hello folks!",
)
print(result.input_tokens)
```

2.5 Streaming (typed semantic events)

Gotcha: a streamed response yields *typed events*, not chunks with `.output`. Accumulate text on `response.output_text.delta` events via `event.delta`.

```

full = ""
for event in client.responses.create(
    model="gpt-5.4-mini", input="Explain backprop in 3 sentences.",
    stream=True):
    if event.type == "response.output_text.delta":
        full += event.delta
        print(event.delta, end="", flush=True)
# other events: response.created, response.output_text.done, response.completed

```

3. Function / Tool Calling

Pattern: define a tool schema → the model emits a `function_call` item → you run the function → send the result back as a `function_call_output` linked by `call_id` → the model produces the final answer. Responses API function-call items are **flat** (`.name`, `.arguments`, `.call_id`) — there is no nested `.function` as in Chat Completions.

```

tools = [{
    "type": "function",
    "name": "get_weather",
    "description": "Get the weather for a city.",
    "parameters": {
        "type": "object",
        "properties": {"city": {"type": "string"}},
        "required": ["city"],
    },
}]

resp = client.responses.create(model="gpt-5.4-mini",
                              input=[{"role": "user", "content": "Weather in
Lisbon?"}],
                              tools=tools)

next_input = []
for item in resp.output:
    # NOTE: iterate resp.output, not resp
    if item.type == "function_call":
        args = json.loads(item.arguments)
        result = get_weather(**args)    # run your function
        next_input.append({
            "type": "function_call_output",
            "call_id": item.call_id,    # link the result to the call
            "output": json.dumps(result),
        })

final = client.responses.create(model="gpt-5.4-mini",
                                input=next_input,
                                previous_response_id=resp.id,
                                tools=tools)

print(final.output_text)

```

4. Built-in Tools

The model executes these server-side — just enable them in `tools`.

Tool	Enable with	Use for
<code>web_search</code>	<code>{"type": "web_search"}</code>	Live web answers with citations (annotations)
<code>file_search</code>	<code>{"type": "file_search", "vector_store_ids": [...]}</code>	RAG over uploaded docs / vector stores
<code>code_interpreter</code>	<code>{"type": "code_interpreter"}</code>	Run Python (data analysis, math, files)
<code>mcp</code>	<code>{"type": "mcp", "server_url": ...}</code>	Call external MCP servers / connectors

```
# Minimal agent loop with the built-in web_search tool
resp = client.responses.create(
    model="gpt-5.4-mini",
    input=[{"role": "user", "content": "Latest LLM news from simonwillison.net?"}],
    tools=[{"type": "web_search"}],
)
for item in resp.output:
    if item.type == "web_search_call": print("searched:", item.action)
    if item.type == "message":         print(item.content[0].text) # answer +
citations
```

5. Conversations API — durable server-side state

A Conversation is a long-lived object with its own id. Pass `conversation=conv.id` to `responses.create` and the API **auto-appends both the user input and the assistant output** — don't add them by hand or you double every turn. In items, user turns use `input_text` and assistant turns use `output_text`.

```

conv = client.conversations.create(metadata={"user_id": "u_123"})
client.conversations.retrieve(conv.id)
client.conversations.update(conv.id, metadata={"status": "in_progress"})

# Items CRUD
client.conversations.items.create(conv.id, items=[
    {"type": "message", "role": "user",
     "content": [{"type": "input_text", "text": "Help me set up my account."}]},
])
items = client.conversations.items.list(conv.id)      # paginated
client.conversations.items.retrieve(conv.id, item_id)
# client.conversations.items.delete(conv.id, item_id)

# Stateful turn — API appends user + assistant automatically
resp = client.responses.create(model="gpt-5.4-mini",
                               input="What can you do?",
                               conversation=conv.id)

```

6. AgentKit Stack — when to climb a layer

- **Agents SDK** — multi-agent orchestration in code: agents, handoffs, function tools, guardrails, and tracing. Works with 100+ models. The durable path as Agent Builder/Evals are deprecated.
- **Agent Builder** — visual workflow composer (Start → Agent → tools/guardrails → End), versioning, evals. *Deprecated Nov 30 2026; the node patterns map directly onto the Agents SDK.*
- **ChatKit** — embeddable chat UI (React component / web component) that fronts a workflow: auth, streaming, file upload, tool-call display.
- **Guardrails** — modular open-source safety layer that runs before and after agents: PII detection (block/redact), jailbreak attempts, prompt-injection defense, off-topic and sensitive-data filtering — no changes to agent logic.

7. Quick Reference & Gotchas

- **Model lineup:** `gpt-5.4-mini` for most work; `gpt-5.5` for frontier reasoning. Responses report a dated snapshot (e.g. `gpt-5.4-mini-2026-03-17`).
- **Iterate `resp.output`, never `resp`:** a `Response` is a Pydantic model — iterating it yields field tuples, not output items.
- **Streaming = events:** accumulate `event.delta` on `response.output_text.delta`; there is no `chunk.output`.
- **Function calls are flat:** `item.name` / `item.arguments` / `item.call_id`; feed results back as `function_call_output`.
- **Conversations auto-append:** with `conversation=...` the API stores both sides of the turn for you.
- **Structured output:** prefer `responses.parse(text_format=Model)` → `output_parsed` over hand-rolled JSON schemas.